

State Coherence Under Partial Observation

A research-backed rewrite of the JOAT Labs pm-substrate thesis, with
ArrowHedgeLabs as the sandbox testbed

Prepared for Emmanuel Akinwale and JOAT Labs

May 26, 2026

Purpose: replace the thesis draft with a research-backed thesis, implementation test plan, plug-in strategy, risk register, and sandbox plan.

Executive Thesis

The modern workspace is not failing because teams lack software. It is failing because work now moves through many specialized tools, teams, and agents that each hold a partial model of the same changing reality. The hard problem is state coherence: how bounded actors can act safely when every actor sees only part of the system, every observation can be stale, and every action may change the state other actors depend on.

JOAT Labs should frame pm-substrate as the project-manager layer for that problem. It is not another app and not a point-to-point integration hub. It is a tenant-scoped operational substrate that coordinates humans, tools, workflows, and AI agents through typed entities, relationships, events, capabilities, policies, workflow state, evidence, projections, commitments, and continuity records.

The AI state problem is therefore not separate from the workspace gap. It is a more visible version of the same systems problem. An agent is a bounded perception-action system: it observes a partial world, encodes those observations into an internal model, chooses actions from that model, and receives delayed or incomplete feedback. Humans, departments, SaaS tools, and LLM agents all share this structure. The difference is that LLM agents make the failure observable because they act quickly across many systems without naturally owning institutional memory.

Rewritten claim: pm-substrate does not solve reality. It solves governed operational state. It records what the workspace accepts as actionable, where that claim came from, how fresh it is, which actor may change it, which workflow it belongs to, and how agents can resume from it after context loss.

What Changed From The Earlier Thesis

The earlier draft was directionally strong because it identified state as the hidden bottleneck behind both AI agents and cross-tool project management. The rewrite sharpens that claim in four ways.

- First, it separates reality state, observed state, belief state, operational state, authoritative state, historical state, and projection state. This avoids claiming that the substrate knows reality directly.
- Second, it makes AI a semantic I/O and mapping assistant, not the source of authority. The LLM proposes mappings and transformations; deterministic validators, profiles, permissions, and write gates decide admissibility.
- Third, it makes plug-in adoption measurable. A platform sits on top of pm-substrate when it can map into the substrate with new mapping/profile/capability files, not by rewriting the substrate or hard-coding itself into existing providers.
- Fourth, it moves ArrowHedgeLabs from an example to a sandbox. The financial research agents already expose the exact problems the substrate should solve: many specialist agents, shared state, risk constraints, workflow edges, evidence, replay, and amnesiac continuity.

The First-Principles Problem

At the lowest useful level, an agent is not a chatbot. It is a stateful system coupled to an environment. The environment has state S . The agent receives observations O , maintains an internal model M , updates that model as observations arrive, chooses an action A through a policy or objective, and then

receives feedback from the changed environment. This structure appears in control systems, animals, humans, departments, software services, robots, and LLM agents.

Once the agent is decomposed, the state problem appears at every layer: observation can be partial, representation can lose structure, memory can become stale, goals can conflict, plans can be invalidated by new events, tools can mutate reality, feedback can be delayed, and identity can disappear across sessions. The LLM context window is only one surface of this deeper problem.

Physics is useful here as a discipline of humility. Observation does not give unlimited, undistorted access to a system. In operational systems, the analogy is not quantum mechanics itself; the analogy is the discipline of admitting that measurement is partial, instruments have side effects, and a system's state must be estimated from bounded observations. pm-substrate should not pretend to hold perfect truth. It should make observation, evidence, uncertainty, freshness, authority, and invalidation explicit.

Research Findings That Support The Approach

The thesis is strongest when grounded in work from several fields. The Arrowsmith or literature-based discovery move is to look for nearby domains that solved pieces of the same abstract problem, then compose the useful mechanisms without pretending any one domain solved the whole workspace.

Finding	Research or standard	Design consequence for pm-substrate
Autonomous agents require profile, memory, planning, and action components.	Wang et al. survey LLM-based autonomous agents around profile, memory, planning, and action [R1].	Agent memory must be a substrate layer above events and workflows, not just prompt history.
Complex LLM workflows benefit from explicit states and transitions.	StateFlow proposes state-driven workflows for LLM task solving [R2].	pm-substrate should expose workflows as durable state machines with valid transitions.
Multi-agent systems need infrastructure for interaction, memory, and coordination.	Li et al. survey LLM multi-agent systems through workflow, infrastructure, and challenges [R3].	Agents should coordinate through shared substrate events and capabilities, not hidden chat.
Cross-functional performance depends on inter-team coordination and information elaboration.	Xie et al. link coordination, information elaboration, and team performance [R4].	The substrate's business value is reduced coordination cost, not isolated intelligence.
Knowledge workers switch applications because work is distributed across task surfaces.	Jahanlou et al. characterize task-centric application switching in knowledge work [R5].	The workspace gap is real at the task level; the system must preserve context across tools.
Provenance can be modeled as entities, activities, and agents.	W3C PROV defines a standard provenance model [S1].	Every state claim should link to source, actor, activity, and evidence.
Declarative mappings let existing schemas project into another model.	W3C R2RML maps relational databases to RDF using mapping documents [S2].	pm-substrate should use declarative entity mappings rather than app rewrites.
JSON data contracts can be validated with formal schemas.	JSON Schema defines a declarative vocabulary for annotating and validating JSON [S3].	AI outputs and platform mappings need schema-constrained proposals and validation gates.
Interoperable data needs language-neutral formats.	Apache Arrow specifies a language-independent columnar memory format [S4].	The substrate should treat formats as adapters and preserve logical meaning above file form.

Finding	Research or standard	Design consequence for pm-substrate
Controllers reconcile desired state and current state in loops.	Kubernetes controllers watch state and move current state toward desired state [S5].	Cron jobs and heartbeats should be reconciliation and freshness checks, not the source of truth.
Risk-managed AI requires measurement, governance, and controls.	NIST AI RMF frames AI risk management through govern, map, measure, and manage functions [S6].	The substrate needs measurable risks, authority boundaries, and audit-ready controls.
LLMs can produce schema-constrained outputs, but the schema is the control.	OpenAI Structured Outputs constrain model output to developer-supplied JSON Schema [S7].	AI can propose plug-in mappings, but accepted substrate writes must pass deterministic contracts.

The common pattern is clear. Other disciplines do not say, 'make one giant intelligent thing.' They externalize state into ledgers, models, schemas, events, workflows, controllers, provenance records, and audit trails. pm-substrate is the same pattern applied to agentic workspaces.

The Key Distinction: AI As Semantic I/O, Not Authority

The user's intuition is right: modern AI is unusually good at turning many kinds of input into structured output. Text, images, audio, tables, CSV files, JSON, database schemas, API descriptions, and messy documents can all be interpreted by capable models. That makes AI the best available semantic adapter layer.

But the substrate should not conclude that AI is the schema. The safe architecture is compiler-like. AI reads the source platform's shape and proposes a mapping. Deterministic validators check the mapping against a profile. Dry-run ingestion tests the mapping against sample rows and events. Human or policy approval promotes the mapping version. Runtime adapters apply the mapping. Substrate write gates validate every mutation. Event provenance records what changed and why.

Formula: AI is the semantic mapper. pm-substrate is the type system, compiler, runtime, event ledger, and audit trail.

This avoids the two-state problem. There should not be one independent AI state and one independent project state competing for authority. Agent state should be interpretive and evidence-backed. Project state should be governed and authoritative only after validation. Derived views should carry freshness metadata. When they disagree, the substrate does not ask which story sounds better; it follows source authority, event history, profile rules, and conflict procedures.

How A Platform Plugs In Without Being Rewritten

The implementation baseline already exists in the pm-substrate repository. ADR-0020 defines the declarative entity-mapping format. ADR-0021 adds profile-aware semantic validation. ADR-0022 adds an ingestion adapter that turns a source row and tenant context into graph-ready inputs. That is the start of the easy plug-in story.

- 1 Discover the source. Inspect the platform's database schema, API schema, CSV headers, event payloads, or sample exports.
- 2 Ask AI to propose the mapping. The model proposes entity mappings, field aliases, edges, source references, event types, and capability descriptors.

- 3 Validate structurally. The mapping must match the substrate mapping format: version, profile, entity declarations, identity fields, field maps, schema versions, and edge declarations.
- 4 Validate semantically. The mapping must resolve against the installed profile: concrete type exists, tier-1 binding matches, required identity fields are covered, edge types and cardinalities match.
- 5 Dry-run sample data. Apply the mapping to representative rows and events. The adapter returns graph-ready node and edge inputs without mutating production state.
- 6 Run write-gate tests. The graph, event log, registry, permissions, workflow runtime, and provenance rules accept or reject the proposed writes deterministically.
- 7 Version and approve. Store the mapping version, sample fixtures, expected outputs, provenance policy, and rollback path.
- 8 Operate through adapters. The source platform remains itself. It only needs an export, API, webhook, CDC stream, or small adapter. The substrate is not rewritten to fit it.

The acceptance criterion is concrete: onboarding a new platform should require new mapping files, fixtures, tests, and maybe a thin adapter, but zero changes to substrate packages and zero changes to existing providers. If a new platform requires changing the graph, event log, registry, workflow runtime, or another provider, the plug-in claim failed.

Dynamic And Structurally Correct

Dynamic does not mean schema-less. It means the system can accept new source shapes while preserving explicit contracts. A platform can bring CSV, JSON, SQL rows, API payloads, documents, or event streams. AI can infer what each field likely means, but the substrate only accepts a mapping once it compiles against a known profile and passes sample-based tests.

Layer	Dynamic behavior	Correctness control
File and API intake	Accept CSV, JSON, SQL, webhooks, APIs, documents, and multimodal evidence.	Use parsers, MIME/type detection, checksums, sample fixtures, and source metadata.
Semantic interpretation	AI proposes entities, relationships, field aliases, and events.	Use structured outputs, mapping schemas, confidence thresholds, review queues, and rejected-output logs.
Profile binding	Different vertical profiles can define concrete types over shared Tier-1 primitives.	Run profile-aware semantic validation before any write path.
Runtime ingestion	Adapters transform source records into graph/event inputs.	Write gates enforce tenant, identity, schema version, permissions, idempotency, and provenance.
State propagation	Events update projections, workflows, and agent continuity.	Replay tests, causation chains, freshness metadata, and contradiction detection.
Evolution	Mappings can change as source platforms change.	Version mappings, migration plans, backfills, compatibility tests, and rollback paths.

This is how the substrate stays both flexible and structurally correct. AI handles semantic interpolation. The substrate handles contracts. That split is the architecture.

Heartbeats, Cron Jobs, And The Observation Problem

The periodic heartbeat idea is useful, but it should be framed carefully. A cron job should not be treated as truth. It should be treated as a reconciliation event: a scheduled opportunity to observe source systems, refresh projections, detect drift, expire stale claims, and force agents to rebase their local plans against the current substrate.

Borrow the Kubernetes controller pattern: watch desired state and current state, then reconcile differences. Borrow observability discipline from OpenTelemetry: traces, metrics, and logs are separate signals that help explain system behavior. Borrow distributed-systems discipline from Lamport: ordered events and causal relationships matter when components do not share a single clock.

In pm-substrate, a heartbeat should emit a state census. The census does not claim perfect reality. It says: these sources were checked, these facts were fresh, these projections were invalidated, these workflows were blocked, these agents have open plans depending on changed state, and these conflicts need human or policy resolution.

Why ArrowHedgeLabs Is The Right Sandbox

ArrowHedgeLabs is a strong sandbox because it already looks like a multi-agent organization. The local project contains multiple specialist analyst agents, a risk manager, a portfolio manager, a LangGraph workflow, structured state, and a backtesting orientation. It also contains Dexter, a research-agent environment with scratchpad, memory, compaction, and tool execution. That makes it close to the problem pm-substrate claims to solve.

Sandbox feature	Why it matters
Many specialist analysts	Represents cross-functional departments with different beliefs and evidence.
Risk manager	Represents deterministic constraints that should gate agent proposals.
Portfolio manager	Already separates LLM judgment from deterministic allowed-action computation.
LangGraph state	Shows current local state shape: messages, data, metadata, and merged analyst signals.
Backtesting	Provides repeatable historical scenarios for replay and regression testing.
Dexter scratchpad and memory	Shows the difference between local agent notes and substrate-governed continuity.

This sandbox must remain research and education only, not financial advice or trading infrastructure. Its value is that financial research has rich state: tickers, dates, evidence, analyst signals, risk limits, portfolio constraints, decisions, outcomes, and replay. That richness makes it a demanding testbed for state coherence.

ArrowHedgeLabs Test Plan

The testbed should validate two claims at the same time: first, that existing platforms can plug into the substrate through mappings and adapters; second, that agents behave better when they resume from substrate state rather than local chat history.

Test	Procedure	Pass condition
T1: Source mapping	Map ArrowHedgeLabs tickers, portfolio, analyst signals, risk outputs, and decisions into substrate entities/events.	No substrate package edits; mapping validates structurally and semantically.
T2: Event provenance	Each analyst signal becomes an event with source, timestamp, evidence payload, ticker, model/provider metadata, and causation chain.	Every final decision can trace back to its contributing signals and risk state.
T3: Deterministic risk gate	Keep compute_allowed_actions as a deterministic boundary and record its output as validated operational state.	LLM can choose only actions and quantities permitted by the risk gate.
T4: Amnesiac resume	Run session 1, store checkpoints, delete chat context, then run session 2 from tenant, agent, and scope only.	Agent resumes open work, avoids contradicted claims, and cites substrate evidence.
T5: Staleness failure	Replay a case where price or position state changes after analyst signals are created.	Workflow blocks or requests refresh before producing a stale decision.
T6: Plug-in file format	Ingest the same scenario from JSON, CSV, and a SQL-like row export.	All formats produce equivalent canonical substrate entities/events through mappings.
T7: Replay audit	Replay a historical backtest from event history and compare projected portfolio decisions.	Replay reproduces decisions or explains deterministic differences from changed mappings/profiles.
T8: Conflict handling	Create conflicting analyst claims and stale risk data.	System records conflict, prevents silent promotion, and routes to rule or human review.

Metrics To Monitor

The substrate should be judged by behavior, not elegance. These metrics tell whether it is solving state coherence and plug-in adoption rather than simply storing more data.

Metric	Definition
Time-to-plugin	Elapsed time from first source schema/sample to validated mapping, dry-run, and first accepted substrate write.
Substrate edit count	Number of changes required in substrate packages to onboard a new platform. Target: zero.
Mapping coverage	Percent of source entities, events, and required fields mapped to Tier-1/profile concepts.
Validator rejection rate	Percent of AI-proposed mappings rejected by structural, semantic, or sample-based validation.
Evidence coverage	Percent of authoritative state transitions linked to source evidence and actor/activity provenance.
State disagreement rate	Frequency of conflicts between source systems, projections, and agent beliefs.
Stale action rate	Percent of attempted actions based on expired or invalidated state.
Agent resume success	Percent of amnesiac runs that resume correct scope, open work, constraints, and evidence without chat history.
Replay fidelity	Percent of historical workflows whose outputs can be reproduced or explained from event history.
Unauthorized action block rate	Blocked attempts that would have mutated state without permission or valid workflow position.
Cross-tool outcome success	Percent of workflows completed across multiple providers without provider-to-provider hard coupling.
Mean time to reconcile	Elapsed time from conflict or drift detection to accepted resolution.

Risks And Mitigation Strategy

Risk	Why it matters	Mitigation
AI hallucinated mapping	The model may infer a field meaning incorrectly.	Use schema-constrained outputs, sample fixtures, deterministic validators, human approval for promotion, and regression tests.
Two-state problem	Agent memory can diverge from project state.	Make agent memory subordinate: claims must cite evidence and rebase against substrate state before action.
Source authority ambiguity	Two tools may disagree about the same fact.	Maintain source-of-truth registry by fact type and conflict resolution policy.
Schema drift	Source platforms change fields or semantics.	Version mappings, monitor ingestion errors, run nightly schema diff, and require migration plans.
Over-instrumentation	Observation itself can change behavior or create noise.	Separate telemetry from authority and use heartbeats as reconciliation, not truth.
Workflow brittleness	Hard-coded flows break when capabilities change.	Use capability descriptors, workflow soundness checks, optional providers, and dead-letter paths.
Privacy and tenant leakage	Cross-tenant operational state is sensitive.	Tenant-scoped writes, permission gates, provenance, audit logs, and least-privilege capability registration.
Financial misuse in sandbox	A trading-like demo can be mistaken for investment advice.	Keep ArrowHedgeLabs as historical/research simulation with no real-trading path.

Implementation Roadmap

- 1 Define the ArrowHedgeLabs research profile: Ticker, ResearchRun, AnalystSignal, RiskState, PortfolioState, Decision, EvidenceDocument, and BacktestRun over existing Tier-1 primitives.
- 2 Write the first ArrowHedgeLabs entity mapping and fixtures. Validate it using the existing structural and semantic validators.
- 3 Add event contracts for analyst.signal.created, risk.state.validated, portfolio.decision.proposed, portfolio.decision.accepted, and workflow.blocked.stale_state.
- 4 Build a read-only ingestion adapter that projects one historical ArrowHedgeLabs run into the substrate without mutating the source project.
- 5 Record deterministic risk-gate output as operational state and require decisions to cite it.
- 6 Store continuity checkpoints for analyst conclusions, risk decisions, unresolved questions, and handoffs.
- 7 Run the amnesiac-agent evaluation using ArrowHedgeLabs: delete chat history and force the agent to resume from substrate continuity plus event history.
- 8 Add the plug-in acceptance test: onboarding ArrowHedgeLabs must not modify substrate packages or existing providers.

This roadmap validates the original PM-layer baseline. The substrate first proves it can coordinate independent providers through mappings, capabilities, events, and workflows. Then the agentic layer

proves the same coordination substrate improves agent continuity, state freshness, and evidence-backed action. The agentic state thesis is therefore not a pivot. It is a harder validation surface for the original project-manager-layer thesis.

Conclusion

The pm-substrate thesis becomes strongest when stated simply: work fails when bounded actors act from divergent local state; the project-manager layer is the governed operational substrate that keeps those actors aligned. AI makes the opportunity larger because it can translate across messy inputs, formats, and schemas. AI also makes the risk larger because it can act quickly from partial or stale state.

The solution is not to make the model the source of truth. The solution is to make the substrate the governed state plane and use AI as the semantic adapter into it. That is how a platform can sit on top of pm-substrate without being rewritten: map it, validate it, dry-run it, version it, and let the substrate enforce the runtime contract.

ArrowHedgeLabs is the right sandbox because it is already a miniature cross-functional organization of agents, constraints, evidence, workflows, and decisions. If pm-substrate can make that system replayable, evidence-backed, resumable, and plug-in clean without rewriting the substrate, it will have validated the core JOAT Labs claim: design the interactions first.

References

- [R1] Wang, L. et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*. <https://link.springer.com/article/10.1007/s11704-024-40231-1>
- [R2] Wu, Y. et al. (2024). StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows. *COLM 2024*. <https://openreview.net/forum?id=3nTbuygoop>
- [R3] Li, X. et al. (2024). A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*. <https://link.springer.com/article/10.1007/s44336-024-00009-2>
- [R4] Xie, X., Ling, C.-D., Liu, W., & Wei, J. (2022). Inter-team coordination, information elaboration, and performance in teams. *Journal of Business Research*. <https://doi.org/10.1016/j.jbusres.2022.05.002>
- [R5] Jahanlou, A. et al. (2023). Task-Centric Application Switching. *Graphics Interface*. <https://www.research.autodesk.com/publications/task-centric-application-switching/>
- [R6] Hatalis, K. et al. (2024). Memory Matters: The Need to Improve Long-Term Memory in LLM-Agents. *AAAI Symposium*. <https://ojs.aaai.org/index.php/AAAI-SS/article/view/27688>
- [R7] Zhang, Y. et al. (2023). Schema Matching using Pre-Trained Language Models. *IEEE ICDE*. <https://www.microsoft.com/en-us/research/publication/schema-matching-using-pre-trained-language-models/>
- [R8] Sebastian, A. et al. (2021). A systematic review of literature-based discovery. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7924697/>
- [S1] W3C. PROV Overview. <https://www.w3.org/TR/prov-overview/>
- [S2] W3C. R2RML: RDB to RDF Mapping Language. <https://www.w3.org/TR/r2rml/>
- [S3] JSON Schema. What is JSON Schema? <https://json-schema.org/overview/what-is-jsonschema>
- [S4] Apache Arrow. Columnar format and language-independent data. <https://arrow.apache.org/docs/format/Intro.html>
- [S5] Kubernetes. Controllers. <https://kubernetes.io/docs/concepts/architecture/controller/>
- [S6] NIST. AI Risk Management Framework. <https://www.nist.gov/itl/ai-risk-management-framework>
- [S7] OpenAI. Structured Outputs. <https://platform.openai.com/docs/guides/structured-outputs>
- [S8] OpenTelemetry. What is OpenTelemetry? <https://opentelemetry.io/docs/what-is-opentelemetry/>
- [S9] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. <https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/>
- [S10] Git. What is Git? <https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>